2 - Design and Implement Comprehensive Test Plans with Instrumented Code

In this document, we want to turn the six requirements from the previous analysis into a test plan. We use an iterative V-Model lifecycle, apply TTD within each coding increment, and identify the logs and controlled environments we need to produce evidence. We should combine references to mathematics, deterministic and statistical checks, integration tests, security-focused tests, representative-user acceptance tests and more.

Priority and Pre-Requisites

As mentioned before, we're working with snnTorch. Here are some assumptions for the plan, and how we treat it:

1. We have exact repository revisions and package versions.
   We treat this by recording commits, any installed package versions, the Python and PyTorch versions themselves, the operating system we're using, and the device we use for every run. We'll confirm these before considering results and tests.
2. We ensure we have supported environments.
   We have to identify the environment that supports Python, PyTorch, CPU and GPU before we can run snnTorch.
3. We have a test runner.
   We'll use the repo's configured test runner. If none is established, we'll choose and document one before the first test-driven development cycle. pytest seems a reasonable candidate for a Python package, and so for snnTorch.
4. We need numerical limits.
   Thus, we set membrane potential and gradient tolerances by dtype and device before running tests.
5. We'll have test data and downloads prepared.
   So, we use synthetic inputs and disposable temporary directories for security and integration tests, and don't use any real user files or depend on live external dataset services for routine sets.
6. We'll have a clear boundary of scope.
   There are many other requirements we could do, but only six have been made explicit in the LO1 analysis as the acceptance baseline. Other goals will be listed as omissions in the Process and Risk section.

The identifiers below are assigned to make the LO1 requirements traceable through the design, coding, testing, and portfolio. The priority P1 or P2 reflects the consequence if the requirement fails.

ID	Requirement and type	Primary level	Priority and acceptance basis
R1	Neuron implementations follow their specified dynamics, including decay, threshold, reset method, and reset timing. Functional and mathematical correctness.	Unit, with selected integration checks	P1. Spikes match the independent reference exactly, and membrane traces match within a pre-agreed dtype/device tolerance.
R2	Surrogate-gradient implementations follow the specified forward spike rule and backward gradient rule. Functional correctness.	Unit and autograd integration	P1. Forward outputs and gradients match independently calculated rules within pre-agreed tolerance, including the defined boundary behaviour.
R3	Rate, latency, and one-hot encoders represent their inputs as specified. Functional correctness, including stochastic behaviour.	Unit with selected workflow checks	P1. Deterministic cases match exactly and intermediate rate probabilities pass a pre-declared statistical acceptance interval.
R4	Independent models do not affect each other unintentionally, and documented reset makes a new sequence behave like a fresh model. State isolation and reliability.	Integration	P1. Fresh-versus-reset and interleaving checks pass across 1,000 declared model/sequence combinations.
R5	A user with basic PyTorch knowledge can install the package and run a minimal supplied SNN example within 30 minutes without instructor help.	System and acceptance	P2. At least 4 of 5 representative users complete the task within 30 minutes.
R6	Dataset extraction cannot write outside its destination, and downloads preserve secure transport behaviour.	Unit, integration, and system	P1. Traversal and escaping-link fixtures are rejected without changing external canaries; HTTPS failure does not cause an insecure downgrade.

For each requirement, we develop a test plan.

R1 Neuron dynamics

For R1, we want to establish that each selected neuron model produces the membrane-state and spike sequence defined by its mathematical specification. The requirement is high priority because small errors in threshold or reset timing can change an experiment's results.

- Design partitions:
  - Each implemented reset method
  - Reset before/after spike as specified
  - Inputs below, at, and above threshold
  - Scalar and vector parameters
  - Single-step and multi-step sequences
  - Batch dimensions
  - Supported dtypes/devices.
- Oracle:
  - Implement the equations independently from the production code, preferably with a small transparent reference function.
  - Have the equations and edge-case semantics checked against the model specification by a knowledgeable reviewer.
  - Keep hand-calculated short traces as anchor cases.
- Pass condition:
  - Compare each spike exactly.
  - Compare membrane values after each time step within a tolerance fixed for the dtype/device before execution.
- Evidence:
  - Requirement ID
  - Model and reset configuration
  - Input sequence
  - Expected and observed trace
  - dtype/device
  - Tolerance
  - Test results
  - Defect link

Lifecycle point	Planned work and evidence
requirement and risk review +test design and scaffolding	Confirm the model equations and reset timing, make threshold inclusivity explicit, and review the independent oracle and partition table before production changes.

Lifecycle point	Planned work and evidence
test-first component work, using TDD	Add failing component tests for each partition, then implement. Capture per-time-step membrane values and spikes. Add every discovered defect as a focused regression.
integration	Run selected tests through a minimal recurrent PyTorch model to catch shape, device, or state-update errors that a direct neuron call would miss.
system and acceptance testing +release and feedback	Run a small end-to-end model smoke trace on each declared supported environment. Keep unit tests as the main proof; do not treat the smoke test as a replacement for boundary coverage.

R2 Surrogate-Gradient Rules

We wish to verify both sides of the surrogate rule. The forward path may use a hard spike while the backward path deliberately uses a different surrogate derivative, so those contracts must be tested independently.

- Design partitions:
  - negative, zero, and positive inputs
  - values around the threshold and any surrogate support boundaries
  - batches
  - parameter setting
  - exact behaviour at points where the derivative is defined specially.
- Oracle and method:
  - calculate forward spikes and backward derivatives from the written formulas independently
  - Use PyTorch gradient extraction (for example, gradients of outputs with respect to inputs and trainable parameters) and compare them with those values.
- Pass condition:
  - forward spike outputs match exactly
  - gradients match the formula within pre-agreed numerical tolerance
- Evidence:
  - formula version
  - input/parameter values
  - expected and observed gradient tensors
  - tolerance
  - PyTorch version/device
  - test result

Lifecycle point	Planned work and evidence
requirement and risk review +test design and scaffolding	Write separate forward and backward contracts, including threshold semantics, and derive small hand-checkable cases and have the reference checked.
test-first component work, using TDD	Introduce failing tests for forward values and gradient values separately, add batch and parameter partitions, and verify that the tests detect deliberate sign, scale, and threshold mistakes in the reference fixture.
integration	Pass the surrogate operation through a short autograd graph and check that gradients reach the expected inputs/parameters without unintended detachment.
system and acceptance testing +release and feedback	Run one minimal optimisation-step smoke test and retain loss/gradient finiteness.

R3 Spike encoders

For R3, we want to verify that rate, latency, and one-hot encoders preserve the intended meaning of their inputs. The rate encoder needs statistical evidence for intermediate probabilities, and the other specified cases can be checked directly.

- Rate encoding:
  - check probabilities 0 and 1 exactly
  - For intermediate probabilities, set the sample size N, significance level alpha, and acceptance interval before observing results
  - Record the observed spike count and the chosen interval
  - Seed the generator for repeatability, and include additional preselected seeds in a slower regression run to reduce dependence on one favourable sequence.
- Latency encoding:
  - Compare each encoded spike time against a separately calculated expected time for representative values, including valid boundaries.
  - Test output shape and time-step indexing explicitly.
- One-hot encoding:
  - verify class preservation for repeated and shuffled labels
  - output shape
  - the expected location of each active entry.
  - Include declared invalid label/shape cases if the API promises rejection.
- Evidence:
  - Encoder/API version
  - input

    - seed, N and alpha for statistical checks
    - expected/observed counts or values
    - test result
    - environment

Lifecycle point	Planned work and evidence
requirement and risk review +test design and scaffolding	Specify input domains, time-step convention, output shapes, and stochastic test parameters, create exact reference vectors.
test-first component work, using TDD	Test deterministic cases first, then add the pre-registered statistical test for intermediate rates, keep statistical failures reproducible and report the observed count rather than only pass/fail.
integration	Feed outputs into a small neuron/model call to reveal incompatible shape, dtype, or time-axis conventions.
system and acceptance testing +release and feedback	Run an encoder-to-model smoke workflow and retain seeds and outputs.

R4 Model-state isolation and resetting

We want to detect order-dependent behaviour, unintended shared/global state, and incomplete reset behaviour. In our LO1 analysis, we proposed 1,000 declared sequence/model combinations.

- For each declared case:
    - build identical models and deterministic inputs
    - compare sequence B after the documented reset of a model that processed sequence A against sequence B on a fresh model.
- Interleave two models, reset one, and verify that the other model's state and subsequent outputs remain unchanged. Include the class-level/global reset paths named by the analysis and record which objects the reset is expected to affect.
- Distribute the 1,000 combinations across reset modes, sequence lengths, batch shapes, and model pairings that are supported by the package. Freeze and retain the generator seed and case manifest. Compare exactly in a deterministic same-device run or use an agreed numerical tolerance where the execution environment requires it.
- Evidence:
    - case ID
    - model identities
    - sequence IDs
    - reset calls
    - state snapshots

- expected and observed outputs
- seed
- environment
- failure reproduction instructions

Lifecycle point	Planned work and evidence
requirement and risk review +test design and scaffolding	Clarify documented reset semantics and expected scope, enumerate case factors and freeze the 1,000-case manifest before examining results.
test-first component work, using TDD	Test individual reset utilities against their documented state effects, test that a reset leaves unrelated state untouched.
integration	Run fresh-versus-reset and interleaved two-model checks across the 1,000 manifest cases, emphasizing global registries and class-level reset behaviour.
system and acceptance testing +release and feedback	Exercise reset as a user would in a minimal workflow. Preserve every failing schedule as a small deterministic regression case.

R5 Beginner installation and example workflow

For r5, we validate the actual beginner experience rather than infer usability from code review. The supplied acceptance rule is at least four of five users completing the task within 30 minutes without an instructor intervening.

- Participants:
  - five people with basic PyTorch knowledge who are not given step-by-step help
  - record eligibility only to the extent needed to confirm the stated profile.
- Task:
  - using the supplied README/examples, install the package in a clean environment and run one minimal SNN example
  - Freeze the starting conditions and task instructions so sessions are comparable
- Record completion, elapsed time, errors, requests for help, and the point of failure. Classify install/dependency/environment problems separately from unclear conceptual instructions, but count them in the overall completion outcome if the requirement says the user must complete installation and run the example.
- Pass condition: at least 4/5 users complete within 30 minutes without instructor intervention. The small sample supports this stated course criterion but does not estimate the experience of the full user population.

Lifecycle point	Planned work and evidence
requirement and risk review +test design and scaffolding	Predefine participant profile, task, clean-machine conditions, start/stop rules, and observation form, pilot the task only to remove procedural ambiguity, not to coach participants.
integration	Run a scripted installation and example smoke check in the same clean environment class before user sessions, capture package and dependency versions.
system and acceptance testing	Conduct five observed task sessions without instructor intervention, report every outcome and timing, including unsuccessful sessions.
release and feedback	Turn observed documentation or install defects into actions and rerun after material changes. Keep original and repeat results distinguishable.

R6 Dataset extraction and secure download behaviour

For R6, we test the security properties stated in the LO1 analysis at unit, integration, and system levels. The test harness must be isolated so a defect cannot overwrite real user files.

- Create harmless archive fixtures containing parent-directory paths, absolute paths, and links that would escape the extraction root. Extract only inside a disposable temporary directory. Place canary files outside the destination and confirm their contents and metadata remain unchanged.
- Pass condition for malicious fixtures:
  - reject unsafe members or otherwise prove they cannot create or alter anything outside the declared destination
  - record the resolved path of each accepted member
  - include both rejection assertions and external-canary checks.
- Simulate HTTPS validation failure, timeout, and unavailable endpoint using a controlled stub or local test service. Assert that the download fails closed or retries only through another verified secure endpoint.
- Inspect the extraction and download code for path normalisation, symbolic-link handling, destination containment, TLS verification, redirects, and error handling.
- Evidence:
  - archive fixture ID
  - destination
  - canary state before/after
  - transport scenario
  - redirect/fallback log
  - package version
  - test result

- security review notes.

Lifecycle point	Planned work and evidence
requirement and risk review +test design and scaffolding	Write a compact threat model for path traversal, absolute paths, escaping links, TLS failure, and insecure fallback. Build fixtures in temporary storage and verify the harness itself is isolated.
test-first component work, using TDD	Test path rejection/containment helpers and secure transport failure behaviour with controlled inputs. Keep fixtures inert; do not execute archived content.
integration	Test the connected download-to-extraction path using a stub/local service and a disposable destination; check that failures leave no partial unsafe state.
system and acceptance testing +release and feedback	Run the full workflow only in an isolated temporary environment; rerun known attack regressions whenever extraction or transport code changes.

Scaffolding and Instrumentation

Instrumentation is the code, test harness, controlled data, and recording needed to make observable and repeatable outcomes. Below, we have scaffolding that is proportionate to the selected requirements, as well as the limitations of each result.

Requirement	Instrumentation and scaffolding	Assessment and improvement
R1	Independent equation-based reference, fixed input traces, trace capture of membrane values and spikes at each time step, fixtures for reset, threshold, shape, dtype, and supported device cases.	Adequate for deterministic model behaviour if the reference is independently reviewed and tolerances are frozen. Could improve by retaining hand-calculated anchor cases and adding the declared supported-device matrix.
R2	Independent forward/backward formulae, gradient capture for inputs and parameters, fixed tensors and parameter values, and autograd integration fixture.	Adequate for the specified derivative if formula and kink behaviour are explicit. Finite differences of the hard spike are not adequate. We improve with mutation checks or deliberately wrong local variants to show the tests are sensitive.

Requirement	Instrumentation and scaffolding	Assessment and improvement
R3	Explicit random generator/seed, exact vectors for latency and one-hot, rate-count collector, predeclared N, alpha, and acceptance interval, and output-shape assertions.	Adequate if statistical parameters are chosen in advance and false rejection risk is stated. Could improve with several preselected seeds in slower regression and report counts/intervals, not only booleans.
R4	Model factory, fresh reference instance, reset/interleaving scheduler, 1,000-case manifest, state snapshots, deterministic input generator, and failure replay record.	Strong for order dependence in the declared cases, but the count does not represent all workloads. Improve by deriving partitions from documented use and retaining every counterexample as a regression.
R5	Clean virtual environment or equivalent, fixed task sheet, clock, completion/time/error/help observation log, scripted environment metadata capture.	Directly observes the target user task. Five participants match the stated decision rule but are insufficient for broad population claims. Improve with later sessions across different experience levels and platforms.
R6	Disposable temporary directories, malicious archive fixtures, external canaries, controlled download/HTTPS stub; redirect and TLS verification logs, and code-review checklist.	Good for specified attack classes and safe to run. Not a complete security assurance. We improve with broader archive fuzz/property tests, dependency review, and a wider secure-transport test matrix if scope permits.
Cross-cutting	Requirement-to-test ID map, configured test runner, CI result logs, environment metadata, seed/tolerance records, defect links, versioned test plan.	This makes evidence traceable and repeatable. Line coverage alone cannot show that each requirement is satisfied. We can improve by auditing requirement-to-test links at each increment and keeping the support matrix explicit.

We record evidence for each test.
To do this, we retain for each executed test a compact record with the following fields:

- Test ID and linked requirement ID(s), test level and lifecycle stage.

- Test code/version, package commit, test-data or fixture ID, and any oracle/specification version.
- Python, PyTorch, operating system, CPU/GPU, dtype, relevant configuration, seed, tolerance, and statistical parameters.
- Expected result, observed result, pass/fail status, captured trace or log location, and defect/retest reference.
- For user sessions:
  - participant profile check
  - start/end time
  - completion status
  - errors
  - help requests.
- For security cases:
  - temporary destination
  - fixture path
  - canary state before/after
  - observed transport/fallback behaviour.

Process and Risk

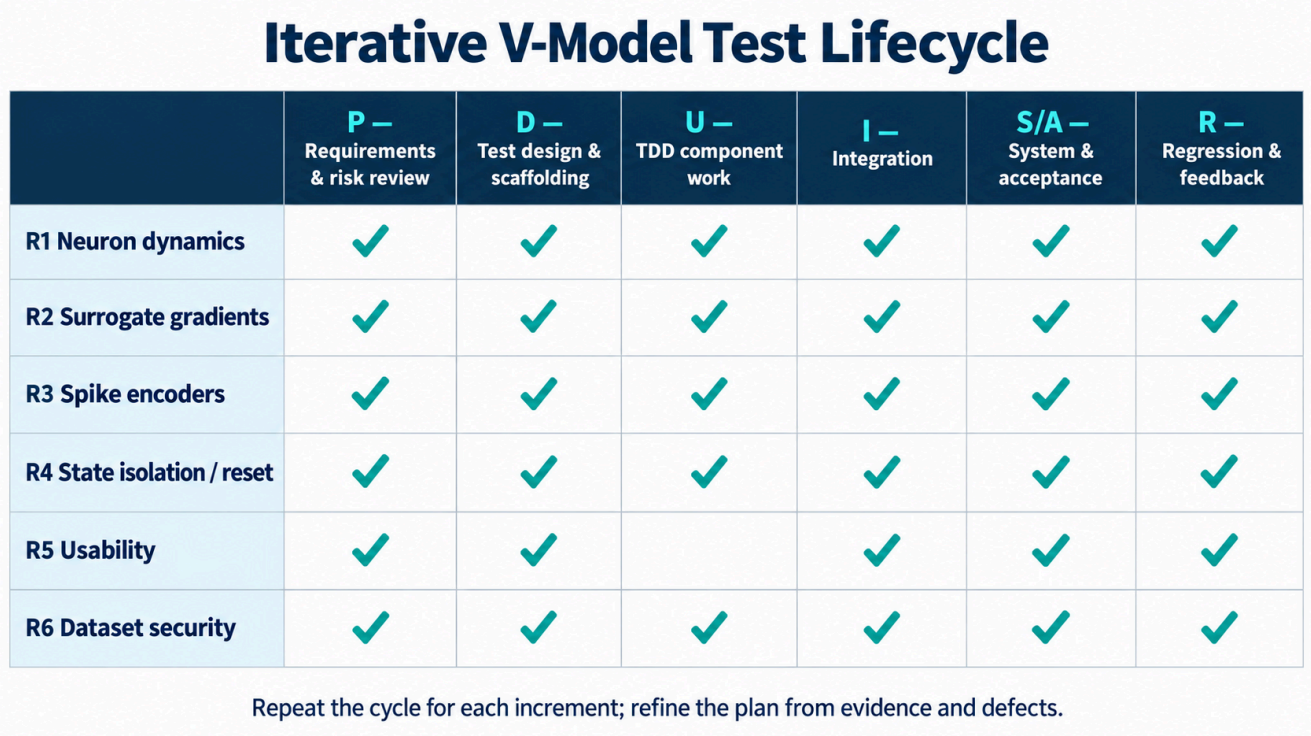
For our lifecycle, we will use an iterative V-cycle, whereby each increment begins with a requirement and its test design, moves through design and implementation, and then verifies the increment at the corresponding unit, integration, system, or acceptance level.

Lifecycle stage	Purpose	Typical outputs and gate
1. Requirement and risk review	Confirm requirement wording, stakeholder need, level, priority, and observable pass condition.	Requirement-to-test trace, unresolved assumptions recorded, no untestable acceptance language silently accepted.
2. Test design and scaffolding	Partition cases, define independent oracles, create controlled data and environments, and review threats.	Reviewed test cases, expected values, seeds, tolerance/statistical parameters, and fixtures available before implementation.
3. Test-first implementation	For each small code change, write a failing test, implement the smallest change that satisfies it, then refactor while keeping tests green.	Component tests and regression evidence linked to requirement IDs, change rationale retained.
4. Integration	Check interactions with PyTorch autograd, model state, package	Cross-component test results, environment metadata, and defect records.

Lifecycle stage	Purpose	Typical outputs and gate
	components, and the dataset transport/extraction boundary.	
5. System and acceptance	Run complete package workflows and representative-user tasks in clean environments.	End-to-end workflow records and the R5 task outcomes.
6. Release and feedback	Run the required regression set, preserve results, and feed defects or new usage evidence into the next increment.	Versioned evidence ledger, regression tests for confirmed defects, and updated plan and risk assessment.

TDD means the exact implementation sequence cannot be fully fixed in advance,and a failing test may expose an issue. The requirement and test intent remain controlled, while test cases and scaffolding may be refined as the code is explored. Each refinement must update the traceability record, state why the previous design was inadequate, and preserve the original acceptance condition unless stakeholders formally change it.

The practical TDD loop is to first choose one requirement partition, then write the smallest failing test, then implement, run the focused test, run nearby regressions, review the result, then proceed to the next partition. Statistical tests and acceptance sessions are planned before their outcomes are observed, even though their final execution occurs later in the lifecycle.



For one person, a small test build should take about 4–6 working days, assuming the package and test runner already work. We'll allow about half a day to confirm the requirements and environment, half a day for shared fixtures, 1½–2½ days for the R1–R4 and R6 tests, half a day for integration and system checks, half a day for the five usability

sessions, and about a day for the final rerun and evidence. The user sessions can be booked while the tests are being built. Major package fixes or missing setup would add time.

The main things that could slow us down are unclear pass conditions, environment setup, booking the five users, and defects that need more than a small fix. We can reduce rework by agreeing the test settings first, recruiting users early, running quick checks after each change, and keeping about a week for fixes and reruns. The state and security tests cover the cases we choose; they cannot prove that every possible case is safe.

The plan only covers the cases we have named, and doesn't show that every model, environment, user, or security attack will behave correctly. We still need to agree the tolerances and statistical settings before testing. Passing these tests supports the six chosen requirements, but doesn't show that training is always reliable or that the package meets a speed or resource target.

Risk or omission	**Effect on confidence**	**Mitigation or required action**
Reference implementation repeats a production mistake.	R1/R2 tests could agree with incorrect code.	Review equations independently; retain hand-calculated anchor cases; separate reference code from production helpers.
Tolerance or supported environment matrix is chosen after results.	Numerical failures could be waived selectively or important devices omitted.	Agree and version tolerances and support matrix before execution; rerun on each declared environment.
Intermediate rate tests have an unsuitable N or alpha.	False rejection or weak detection of probability errors.	Choose sample size/significance/interval before testing; report observed counts; add preselected seeds for slower runs.
1,000 state cases are not representative of real usage.	Some interleavings or reset scopes may remain untested.	Define case factors from documented usage; cover boundary sequence lengths and model pairings; retain every failure as a regression.
Five usability participants provide limited population evidence.	The threshold does not estimate all beginners' experience.	Report individual outcomes, not just 4/5; label the inference limit; expand later if broad usability claims are needed.
Security fixtures cover named attacks but not all archive/parser behaviour.	A novel archive or transport edge case may escape detection.	Combine tests with code inspection; add property-based/fuzzed fixtures and supported archive-format cases where feasible.

Risk or omission	Effect on confidence	Mitigation or required action
Python/PyTorch/device compatibility requirements are not stated.	System tests may pass only on one local setup.	Declare supported versions/devices and run the planned smoke/regression matrix before portfolio conclusions.
General training reliability is not operationalized.	A passing gradient test does not show training converges or remains stable.	If training reliability is in scope, add a separate measurable requirement specifying datasets/workflows, success metric, seeds, and acceptable range.
Resource use/predictable costs have no target.	No defensible performance or cost pass/fail result can be reported.	Define workload, baseline, hardware, latency/memory/compute metric, and acceptable bound before adding a resource requirement.
PyTorch integration is only partly covered by the selected requirements.	Autograd and state tests do not establish compatibility with every PyTorch facility/version.	Add explicit integration requirements if compatibility breadth is an assessment objective; otherwise keep the claim limited to the tested paths.